SEC1_DATA SCIENCE

Eng: mariam alghetany

PANDAS

- Pandas specially working with tabular data that can come in a variety
- of formats. For this purpose.
- it is built around a type of data structure known as DataFrame.
- A Series: is a one-dimensional array of indexed data, the argument index is optional

```
import pandas as pd
s1 = pd.Series(['a', np.arange(6).reshape(2,3), 'b', [3.4, 4]]) # to
create a series from a list
```

```
0                          a
1    [[0, 1, 2], [3, 4, 5]]
2                          b
3                  [3.4, 4]
dtype: object
```

PANDAS

```
1                          a
2    [[0, 1, 2], [3, 4, 5]]
3                          b
4                  [3.4, 4]
dtype: object
```

```
1. s1 = pd.Series(['a', np.arange(6).reshape(2,3), 'b', [3.4, 4]],
index=np.arange(1,5)) # providing the index
```

```
0    2
1    3
2    4
3    5
4    6
5    7
6    8
7    9
```

```
1. s2 = pd.Series(np.arange(2, 10))
2. s2.values
```

```
array([2, 3, 4, 5, 6, 7, 8, 9])
```

PANDAS

```
1. s2.index
```

```
RangeIndex(start=0, stop=8, step=1)
```

```
1. s2[2] # using index 2 to access its associated value
```

```
4
```

```
1. s2[1:-1:2] # the use of slicing
```

```
1    3
3    5
5    7
dtype: int64
```

PANDAS

- the slicing provides a view of the original Series:

1. s3 = s2[2::2]
2. s3[2] += 10
3. s2

```
0     2
1     3
2    14
3     5
4     6
5     7
6     8
7     9
dtype: int64
```

1. s3

```
2    14
4     6
6     8
dtype: int64
```

PANDAS

- An important difference between accessing the elements of Series and numpy arrays is the explicit index used in Series as opposed to the implicit index used in numpy.

1. #s3[0] # raises error if run
2. s3[2] ➔ 14

- if we consider the index as a label for the corresponding value. Thus, when we write s3[2], we try to access the value that corresponds to label =2.

- This mechanism gives more flexibility to Series objects because for the indices we can use other types rather than integers.

PANDAS

```
1. s4 = pd.Series(np.arange(10, 15), index = ['spring', 'river', 'lake', 'sea', 'ocean'])
2. s4
```

```
spring     10
river      11
lake       12
sea        13
ocean      14
dtype: int64
```

➢ We can also access multiple elements using a list of labels

```
1. s4[['river', 'sea']]
```

```
river    11
sea      13
dtype: int64
```

PANDAS

1. s3[0:2]

```
2    14
4     6
dtype: int64
```

1. s4['spring':'ocean':2]

```
spring    10
lake      12
ocean     14
dtype: int64
```

- Notice that when using the explicit index in slicing, both the start and the stop are included
- When in a Series we have integer indexing, the fact that slicing uses implicit indexing could cause confusion.

PANDAS

1. s = pd.Series(np.random.randint(1, 10, 7), index = np.arange(2, 9))
2. s

```
2    2
3    5
4    1
5    2
6    2
7    6
8    9
dtype: int64
```

3. s[0:3] # slicing uses implicit indexing

```
2    2
3    5
4    1
dtype: int64
```

PANDAS

- To resolve such confusions, pandas provides two attributes loc and iloc for Series and DataFrame objects

1. s.iloc[0:3] # note that the value corresponding to "3" is not included

```
2    2
3    5
4    1
dtype: int64
```

2. s.loc[2:4] # note that the value corresponding to "4" is included

```
2    2
3    5
4    1
dtype: int64
```

PANDAS

- we can create Series from dictionaries

```
1. s = pd.Series({"c": 20, "a": 40, "b": 10})
2. s
```

```
c    20
a    40
b    10
dtype: int64
```

```
1. s = pd.Series({"c": 20, "a": 40, "b": 10}, index=["b","c","d","c"] )
2. s
```

```
b    10.0
c    20.0
d     NaN
c    20.0
dtype: float64
```

PANDAS

```
1. s = pd.Series('hi', index = np.arange(5))
2. s
```

```
0    hi
1    hi
2    hi
3    hi
4    hi
dtype: object
```

- A Data Frame is a generalization of Series to 2-dimension; that is, a 2-dimensional data structure where both the rows and columns are indexed (labeled).
- in Data Frame both the rows and columns can have different types

PANDAS

```
1. d = {
2. "one": pd.Series('hi', index=["a", "b", "c", "d"]),
3. "two": pd.Series([1.0, ["yes", "no"], 3.0, 5], index=
   ["a", "b", "c","d"]),
4. "three": pd.Series({"c": 20, "a": 40, "b": 10},index=
   ["a","b","c","d"])}
5. d
```

```
{'one': a    hi
 b    hi
 c    hi
 d    hi
 dtype: object,
 'two': a          1.0
 b    [yes, no]
 c          3.0
 d            5
 dtype: object,
 'three': a    40.0
 b    10.0
 c    20.0
 d     NaN
 dtype: float64}
```

PANDAS

1. df = pd.DataFrame(d)
2. df # pandas nicely presents the DataFrame

	one	two	three
a	hi	1.0	40.0
b	hi	[yes, no]	10.0
c	hi	3.0	20.0
d	hi	5	NaN

➢There are various ways to create a DataFrame

- from a dictionary of Series
- from ndarrays or lists
- from a dictionary of ndarrays or lists
- from a list of dictionaries

PANDAS

```
import numpy as np
import pandas as pd
a = np.array([[10,30,200,1000],[80, np.nan, 300, 900]]).T # to transpose
df = pd.DataFrame(a, index = ['lake', 'pond', 'river', 'spring'],
columns = ['Kazakhstan','Japan'])
df
```

	Kazakhstan	Japan
lake	10	80.0
pond	30	NaN
river	200	300.0
spring	1000	900.0

PANDAS

	Kazakhstan	Japan
spring	100	900.0
river	200	300.0
pond	30	NaN
lake	10	80.0

```
1. dict = {"Kazakhstan": [100, 200, 30, 10], "Japan": [900, 300, None, 80]}
2. df = pd.DataFrame(dict, index=['spring', 'river', 'pond', 'lake'])
3. df
```

```
1. my_list = [{"Kazakhstan": 100, "Japan": 900}, {"Kazakhstan": 200, "Japan": 300}, {"Kazakhstan": 30}, {"Kazakhstan": 10, "Japan": 80}]
2. df = pd.DataFrame(my_list, index=['spring', 'river', 'pond', 'lake'])
3. df
```

PANDAS

1. list1 = [{"Kazakhstan": 100, "Japan": 900}, {"Kazakhstan": 200, "Japan": 300}, {"Kazakhstan": 30}, {"Kazakhstan": 10, "Japan": 80}]
2. df = pd.DataFrame(list1, index=['spring', 'river', 'pond', 'lake'], columns=['Japan', 'USA', 'Kazakhstan'])
3. df

	Japan	USA	Kazakhstan
spring	900.0	NaN	100
river	300.0	NaN	200
pond	NaN	NaN	30
lake	80.0	NaN	10

PANDAS

1. df['Japan']

2. df.Japan

➢Two code have same output

```
spring    900.0
river     300.0
pond        NaN
lake       80.0
Name: Japan, dtype: float64
```

1. df[['USA','Japan']]

	USA	Japan
spring	NaN	900.0
river	NaN	300.0
pond	NaN	NaN
lake	NaN	80.0

PANDAS

1. df.iloc[1:4,0:2]

```
       Japan  USA
river  300.0  NaN
pond     NaN  NaN
lake    80.0  NaN
```

1. df.Japan.iloc[:2]

```
spring    900.0
river     300.0
Name: Japan, dtype: float64
```

1. df.loc[['river','lake'], 'Japan':'USA']

```
       Japan  USA
river  300.0  NaN
lake    80.0  NaN
```

1. df.iloc[1,0]

```
300.0
```

```
#df['river', 'Japan'] # raises error if run
```

PANDAS

- Example:

```
import pandas as pd
dict1 = {
"GPA": pd.Series([3.00, 3.92, 2.89, 3.43, 3.55, 2.75]),
"Name": pd.Series(['Askar', 'Aygul', 'Ainur', 'John', 'Smith',
'Xian']),
"Phone": pd.Series([8728, 8730, 8712, 8776, 8770, 8765]) }
df = pd.DataFrame(dict1)
df
```

	GPA	Name	Phone
0	3.00	Askar	8728
1	3.92	Aygul	8730
2	2.89	Ainur	8712
3	3.43	John	8776
4	3.55	Smith	8770
5	2.75	Xian	8765

PANDAS

1. df.GPA > 3.5

```
0    False
1     True
2    False
3    False
4     True
5    False
Name: GPA, dtype: bool
```

1. df.Name[df.GPA > 3.5]

```
1    Aygul
4    Smith
Name: Name, dtype: object
```

1. df[df.GPA > 3.5]

	GPA	Name	Phone
1	3.92	Aygul	8730
4	3.55	Smith	8770

PANDAS

- There are two handy methods for DataFrame objects, namely, head(n=5) and tail(n=5) that return the first and last n rows of a DataFrame (the default value of n is 5)
- we can access the column and index labels through the columns and index attributes of a DataFrame

1. df.head()

	GPA	Name	Phone
0	3.00	Askar	8728
1	3.92	Aygul	8730
2	2.89	Ainur	8712
3	3.43	John	8776
4	3.55	Smith	8770

1. df.columns

```
Index(['GPA', 'Name', 'Phone'], dtype='object')
```

PANDAS

- A powerful feature of Pandas is its ability to easily read data from and write to a variety of common formats
- As an example, suppose we would like to read the table of pandas I/O tools,

```
url = 'https://pandas.pydata.org/pandas-docs/stable/user_guide/io.html'
table = pd.read_html(url)
print(len(table))
```

```
8
```

PANDAS

- The read_html() returns a list of possible DataFrames in the html file This is why the length of the above table is 8

1. table[0]

	Format Type	Data Description	Reader	Writer
0	text	CSV	read_csv	to_csv
1	text	Fixed-Width Text File	read_fwf	NaN
2	text	JSON	read_json	to_json
3	text	HTML	read_html	to_html
4	text	LaTeX	NaN	Styler.to_latex
5	text	XML	read_xml	to_xml
6	text	Local clipboard	read_clipboard	to_clipboard
7	binary	MS Excel	read_excel	to_excel
8	binary	OpenDocument	read_excel	NaN
9	binary	HDF5 Format	read_hdf	to_hdf
10	binary	Feather Format	read_feather	to_feather
11	binary	Parquet Format	read_parquet	to_parquet
12	binary	ORC Format	read_orc	to_orc
13	binary	Stata	read_stata	to_stata
14	binary	SAS	read_sas	NaN
15	binary	SPSS	read_spss	NaN
16	binary	Python Pickle Format	read_pickle	to_pickle
17	SQL	SQL	read_sql	to_sql
18	SQL	Google BigQuery	read_gbq	to_gbq

EX

Extract the table of datasets for "News articles" only and then find the minimum sample size used in datasets used in this table (i.e., the minimum of column "Instances"). In doing so,

1) Do not extract all tables to choose this one from the list of tables. Make sure you *only* extract this table. To do so, use something unique in this table and use it as for the `match` argument of `pandas.read_html()`;
2) The column "Instances" should be converted to numeric. For doing so, use `pandas.to_numeric()` function. Study and properly use the `errors` parameter of this function to be able to complete this task.
3) See an appropriate `pandas.DataFrame` method to find the minimum.

EX

```
import pandas as pd
# Read the table that specifically contains "News articles" using a unique identifier
df_list = pd.read_html("your_file.html", match="News articles")
# Assuming only one table is matched
df = df_list[0]
# Convert the "Instances" column to numeric, handling errors appropriately
df["Instances"] = pd.to_numeric(df["Instances"], errors="coerce")
# Find the minimum sample size used in the datasets
min_instances = df["Instances"].min()
print("Minimum sample size in News articles dataset:", min_instances)
```

EX

s1
g 11
r 25
p 55
a 10

s2
0 20
1 30
2 40

Exercise 7: We have two `Series` defined as follows: available:

```
import pandas as pd
s1 = pd.Series([11, 25, 55, 10], index = ['g', 'r', 'p', 'a'])
s2 = pd.Series([20, 30, 40])
```

We create a `DataFrame` by:

```
df = pd.DataFrame({"col1": s1, "col2": s2})
```

a) What is `len(df.index)`? Why?

7

EX

Exercise 11: Which of the following options can not create a `DataFrame()`? Explain the reason (assume **pandas** is imported as **pd**).

a)

```
dic = {"A": [10, 20, 30, 40], "B": [80, 90, 100, None]}
pd.DataFrame(dic, index=['z', 'd', 'a', 'b', 'f'])
```

b)

```
dic = {"A": [10, 20, 30, 40], "B": [80, 90, 100, None]}
pd.DataFrame(dic)
```

c)

```
dic = {"A": [10, 20, 30, 40], "B": [80, 90, None]}
pd.DataFrame(dic)
```

d)

```
import numpy as np
dic = {"A": np.array([10, 20, 30, 40]), "B": [80, 90, 100, None]}
pd.DataFrame(dic, index=['z', 'd', 'a', 'b'])
```

EX

Exercise 12: A DataFrame named df is created as follows:

```
list1 = [{1: 'a', 2: 'b'}, {3: 'c'}, {1: 'd', 2: 'e'},]
df = pd.DataFrame(list1, index=[1, 2, 3])
df
```

In each of the following parts, determine whether the indexing is legitimate, and if it is, deteremine the output:

a) df.loc[1,1] a

b) df[1] 2

c) df[1][1] a

d) df[1].iloc[1] nan

e) df[1].loc[1] a

f) df[1,1] syntax error

a	b	nan
2		
3		

TASK

Exercise 2: The "data" folder, contains `GenomicData_orig.csv` file. This file contains a real dataset taken from Gene Expression Omnibus (GEO), https://www.ncbi.nlm.nih.gov/geo/ with accession ID `GSE37745`. The original data contains gene expression taken from 196 individuals with either squamous cell carcinoma (66 individuals), adenocarcinoma (106), or large cell carcinoma (24); however, we have chosen a subset of these individuals and also artificially removed some of the measurements to create the effect of missing data.

TASK

1) Use an appropriate reader function from `pandas` to read this file and show the first 10 rows.
2) What are the labels of the first column (index 0) and 20th column (index 19)? Hint: use the `columns` attribute of the DataFrame
3) drop the first two columns (index 0 and 1) because they are sample ID info and we do not need. Hint: use `pandas.DataFrame.drop()` method. Read documentation to use it properly. You can use Part 2 to access the labels of these columns to use as input to `pandas.DataFrame.drop()`
4) How many missing values are in this dataset?
   Hint: first use `pandas.DataFrame.isnull()` to return a `DataFrame` with `True` for missing values and `False` otherwise, and then either use `numpy.sum()` or `pandas.DataFrame.sum()`. Use `sum()` two times.
5) Use `DataFrame.fillna()` method to fill the missing values with 0 (filling missing values with some values is known as *imputing* missing values). Use the method in Part 4 to show that there is no missing value now.